1. DSA Fundamentals + Complexity (Patterns)

Same Concepts. Different Problems. Infinite Possibilities.

Why Learn DSA Patterns ?

- Helps in solving problems faster
- Common patterns repeat in interviews
- Improves problem solving skills
- Builds foundation for advanced topics

Learn Patterns Not Just Solutions !

"Different problems, Same Patterns !"

Common DSA Patterns (High Level)

1. Brute Force
2. Two Pointers
3. Sliding Window
4. Prefix Sum / Difference Array
5. Binary Search (on Answer)
6. Sorting + Two Pointers
7. Hashing (Maps / Sets)
8. Frequency Counting
9. Stack (Monotonic)
10. Queue / Deque
11. Linked List (Fast & Slow, Reversal)
12. Tree Traversals (DFS / BFS)
13. Binary Search Tree (BST)
14. Heap / Priority Queue
15. Greedy Pattern
16. Divide and Conquer
17. Backtracking
18. Dynamic Programming (DP)
19. Graphs (BFS / DFS / Shortest Path)
20. Interval / Range Problems

Patterns appear everywhere — once you see them, problems get easier !

When to Use Which Pattern ?

Pattern	When to Use ?
Brute Force	Small input size, try all possibilities
Two Pointers	Sorted array, pairs, subarrays
Sliding Window	Subarray / substring with conditions
Binary Search	Sorted data, search space, optimize answer
Hashing	Fast lookups, duplicates, frequency
Stack / Queue	Order based problems (next greater, BFS, etc.)
Tree / Graph	Hierarchical or connected data
Greedy	Local optimal choice gives global solution
Dynamic Programming	Overlapping subproblems + optimal substructure
Backtracking	Explore all possibilities (combinations, permutations)

Key Takeaways

1. Learn and recognize patterns.
2. Practice multiple problems for each pattern.
3. Focus on understanding when to apply a pattern.
4. Most hard problems are just a combination of these patterns.
5. Consistent practice is the key to mastering DSA.

"It's not about remembering solutions, it's about recognizing the pattern."

Practice < Improve < Grow < Crack It!

Same Effort Higher Results !

Think
Analyze
Solve
Repeat

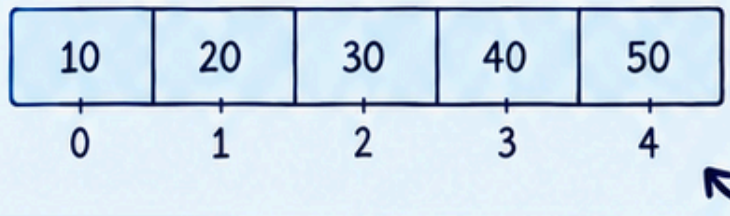
2. Arrays + Hashing

Same Concepts. Different Questions. You Got This !

Same
Concepts
Different
Questions
You Got This !

What is Array ?

An array is a linear data structure that stores elements of the same type in contiguous memory locations. It allows random access using index.



Contiguous Memory
Same Type
Random Access

Common Array Patterns

1. Traversal (find, count, sum, etc.)
2. Find min / max
3. Prefix Sum
4. Two Pointers (on sorted / unsorted array)
5. Sliding Window (subarray problems)
6. In-place modifications
7. Sorting + Searching
8. Frequency Counting (using array / hash map)

What is Hashing ?

Hashing is a technique to store and retrieve data efficiently using a key-value pair. It provides average O(1) time complexity for insert, delete and search operations.

Key (e.g. number) → Hash Function → Index (store value)

Common Hashing Patterns

1. Frequency count (hash map / hash set)
2. Check if element exists
3. Find first non-repeating element
4. Count pairs with given sum
5. Find subarray with given sum
6. Longest subarray with K distinct elements
7. Use hashing to optimize brute force
8. Two sum / Four sum and variations

Common Operations & Complexity

Operation	Array	Hash Map (unordered_map)
Access	O(1)	–
Search	O(n)	O(1)
Insert	O(n)	O(1)
Delete	O(n)	O(1)

When to Use ?

Use Case	Data Structure
Need random access	Array
Need to count frequency / check existence	Hash Map / Hash Set
Need ordered data	Array (with sorting)
Need fast lookups	Hash Map / Hash Set

Example

1. Frequency of elements using Hash Map

```
arr = [1, 2, 2, 3, 1, 4]
freq = {}
for num in arr:
    if num in freq:
        freq[num] += 1
    else:
        freq[num] = 1
print(freq)
# Output: {1: 2, 2: 2, 3: 1, 4: 1}
```

Hashing turns many hard problems into easy ones

Key Takeaways

1. Arrays are simple but very powerful.
2. Hashing helps in fast lookups and frequency counting.
3. Many array problems can be optimized using hashing.
4. Understand when to use array vs hash map.
5. Practice common patterns, they appear frequently in interviews.

Better Code
Better Opportunities
Keep Going :)

Practice
Build
Improve

Same Concepts Different Questions You Got This! ☺

3. Two Pointers

Same Concepts. Different Questions. You Got This !

Practice Solve Improve Grow :)

1. What is Two Pointers ?

- Two Pointers is a technique where we use two indices (pointers) to iterate over a data structure, usually an array or a linked list.
- It helps in solving problems more efficiently (usually in O(n) time) instead of using brute force ($O(n^2)$).
- Pointers can move in the same direction, opposite direction or towards each other depending on the problem.
- Mostly used with sorted arrays or when we need to find a pair/subarray/partition.

2. Types of Two Pointers Patterns

1. Opposite Direction (start & end)
2. Same Direction (fast & slow)
3. Sliding Window (variable size window)
4. Partitioning / Rearranging
5. On Linked List (fast & slow pointers)

3. When to Use ?

- When the problem involves pairs or subarrays.
- When the array is sorted (or can be sorted).
- To reduce time complexity from $O(n^2)$ to O(n).
- When we need to maintain a window or partition.
- Useful in array, string and linked list problems.

4. Common Problems (Two Pointers)

1. Find pair with given sum (sorted array)
2. Check if array is a palindrome
3. Remove duplicates from sorted array
4. Move all zeros to end
5. Three sum (use two pointers after fixing one)
6. Container with most water
7. Trapping rain water (variation)
8. Linked list cycle detection (Floyd's algorithm)

5. Common Templates

(A) Opposite Direction (start & end)

```
left = 0
right = n - 1
while left < right:
    # process nums[left] and nums[right]
    if condition:
        left += 1
    else:
        right -= 1
```

Used in pair sum, palindrome, container with most water, etc.

(B) Same Direction (fast & slow)

```
slow = 0
for fast in range(n):
    # process nums[fast]
    if condition:
        nums[slow] = nums[fast]
        slow += 1
```

Used in remove duplicates, move zeros, linked list cycle, etc.

(C) Sliding Window (variable size)

```
left = 0
for right in range(n):
    # expand window
    while window condition not valid:
        # shrink window
        left += 1
```

Used in subarray problems, longest substring, etc.

6. Example : Pair with Given Sum (Sorted Array)

```
def twoSumSorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1
        else:
            right -= 1
    return []
```

Dry Run:

nums = [1, 2, 4, 7, 11]
target = 9

left ↓				right ↓
1	2	4	7	111

1 + 11 = 12 (> 9) → right--
1 + 7 = 8 (< 9) → left++
2 + 7 = 9 → return [1, 3]

7. Key Takeaways

- ✅ Two Pointers can reduce time complexity to O(n).
- ✅ Identify the type of two pointer pattern.
- ✅ Use sorted array when possible.
- ✅ Practice variations to build intuition.
- ✅ Very common in interviews.

Same pattern appears in many problems, just the logic changes! ☺

Better Code
Better Opportunities
Keep Going :)

Practice
Build
Improve

Think
Analyze
Solve
Repeat

4. Sliding Window

Same Concepts. Different Questions. You Got This !

Practice
Solve
Improve
Grow :)

1. What is Sliding Window ?

Sliding Window is a technique used to solve problems on arrays or strings where we need to find a subarray or substring that satisfies a given condition. Instead of using nested loops ($O(n^2)$), we maintain a window and slide it over the data to get an optimized solution (usually $O(n)$).

Optimize
Problems
Efficiently !

2. Types of Sliding Window

1. Fixed Size Window
   - When the size of the window is constant
   - Eg. maximum sum of subarray of size k
2. Variable Size Window
   - When the size of the window changes based on a condition
   - Eg. longest substring without repeating characters

3. When to Use Sliding Window ?

- Problems involving subarrays or substrings.
- Need to find maximum / minimum / count in a window.
- To optimize from $O(n^2)$ to $O(n)$.
- Works well with arrays and strings.

Right
Technique
Faster
Solutions !

4. Example (Fixed Size Window)

```
// Maximum sum subarray of size k
int maxSum(int[] nums, int k) {
    int n = nums.length;
    int sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];
    int ans = sum;
    for (int i = k; i < n; i++) {
        sum += nums[i] - nums[i - k];
        ans = Math.max(ans, sum);
    }
    return ans;
}
```

Slide,
Update,
Get Result !

5. Common Problems (Sliding Window)

1. Maximum sum subarray of size k
2. Longest substring without repeating characters
3. Longest substring with at most k distinct characters
4. Minimum size subarray sum (>= target)
5. Count subarrays with given sum
6. Find all anagrams in a string
7. Smallest window containing all characters
8. Longest repeating character replacement

General Template (Fixed Size)

```
int n = nums.length;
int k = ...;

// process first window
for (int i = 0; i < k; i++) {
    // add nums[i]
}

// slide the window
for (int i = k; i < n; i++) {
    // add nums[i]
    // remove nums[i - k]
    // update answer
}
```

Process
→ Slide
→ Update
→ Answer

General Template (Variable Size)

```
int left = 0;
for (int right = 0; right < n; right++) {
    // add nums[right]
    // shrink window while condition not valid
    while (/* condition */) {
        // remove nums[left]
        left++;
    }
    // update answer
}
```

Note :

Sliding Window helps us optimize brute force solutions by reusing previous computations instead of recalculating everything.

Better Code
Better Opportunities
Keep Going :)

Small Steps
Big Progress !

Learn
Practice
Implement
Grow :)

5. Prefix Sum + Difference Array

Same Concepts. Different Questions. You Got This !

Small
Steps
Big
Progress !

1. What is Prefix Sum ?

Prefix Sum is a technique where we precompute the cumulative sum of an array, so that we can find the sum of any subarray in O(1) time instead of O(n).

2. Idea

Precompute
Use Anytime
O(1) Queries !

Create an array prefix[] where

```
prefix[i] = prefix[i - 1] + nums[i]
```

prefix[0] = nums[0]

3. Example

nums = [1, 2, 3, 4, 5]

Index (i)	0	1	2	3	4
nums[i]	1	2	3	4	5
prefix[i]	1	3	6	10	15

Sum from l to r = prefix[r] - prefix[l - 1]
(if l > 0)

4. Code (Prefix Sum)

```
int n = nums.length;
int[] prefix = new int[n];
prefix[0] = nums[0];
for (int i = 1; i < n; i++) {
    prefix[i] = prefix[i - 1] + nums[i];
}

// sum of subarray [l, r]
int sum = prefix[r] - (l > 0 ? prefix[l - 1] : 0);
```

5. When to Use Prefix Sum ?

- Find sum of subarrays quickly
- Count subarrays with given sum
- Useful in problems on ranges, frequencies, and cumulative calculations

Faster
Queries
Bigger
Possibilities !

6. What is Difference Array ?

Difference Array is a technique used when we need to perform multiple range updates efficiently. Instead of updating each element in the range, we update only two positions and then compute the final array using prefix sum.

Update
Ranges
Efficiently !

7. Idea

For a range update [l, r] with value val:

```
diff[l] += val
diff[r + 1] -= val    (if r + 1 < n)
```

8. Example

n = 5, operations:
1. add 3 to range [1, 3]
2. add 2 to range [0, 2]

Initial diff = [0, 0, 0, 0, 0, 0] (n + 1 size)

Operation	diff array (n + 1 = 6)
Initial	[0, 0, 0, 0, 0, 0]
After [1, 3] + 3	[0, 3, 0, 0, -3, 0]
After [0, 2] + 2	[2, 3, 0, -2, -3, 0]

Now take prefix sum of diff:
Resultant array = [2, 5, 5, 3, 0]

9. Code (Difference Array)

```
int n = 5;
int[] diff = new int[n + 1];
// range update [l, r] with val
diff[l] += val;
diff[r + 1] -= val;  // if r + 1 < n

// build final array using prefix sum
int[] res = new int[n];
res[0] = diff[0];
for (int i = 1; i < n; i++) {
    res[i] = res[i - 1] + diff[i];
}
```

10. When to Use Difference Array ?

- Multiple range updates
- When we need to add a value to all elements in a range
- Useful in problems like booking systems, frequency updates, etc.

Better Code
Better Opportunities
Keep Going :)

Practice
Build
Improve :)

Think
Analyze
Solve
Repeat :)

6. Binary Search

Same Concepts. Different Questions. You Got This !

Practice
Solve
Improve
Grow !

1. What is Binary Search ?

- Binary Search is a technique to find a target element in a sorted array by repeatedly dividing the search space into half. It works in O(log n) time by comparing the middle element and discarding half of the search space each time.

Divide & Conquer Search Faster !

2. Template (Iterative)

```
int binarySearch(int[] nums, int target) {
   int left = 0, right = nums.length - 1;
   while (left <= right) {
      int mid = left + (right - left) / 2;
      if (nums[mid] == target) return mid;
      else if (nums[mid] < target) left = mid + 1;
      else right = mid - 1;
   }
   return -1; // not found
}
```

3. Template (Recursive)

```
int binarySearch(int[] nums, int target, int left, int right) {
   if (left > right) return -1;
   int mid = left + (right - left) / 2;
   if (nums[mid] == target) return mid;
   else if (nums[mid] < target)
      return binarySearch(nums, target, mid + 1, right);
   else
      return binarySearch(nums, target, left, mid - 1);
}
```

4. When to Use ?

- When the data is sorted
- To find a specific element (exists or not)
- To find the first/last occurrence
- To find insertion position
- To search on the answer space (binary search on range)
- Useful in optimization problems

Sorted Data
Faster Search
Better Solutions!

5. Common Binary Search Patterns

1. Find element in sorted array
2. Find first occurrence of target
3. Find last occurrence of target
4. Find number of occurrences
5. Find insertion position
6. Search in rotated sorted array
7. Binary search on answer (search space)
8. Find minimum/maximum in a sorted range

Same Pattern
Many Problems !

6. Variations

1. First Occurrence (lower bound)
2. Last Occurrence (upper bound)
3. Search in Rotated Sorted Array
4. Find Peak Element
5. Find Minimum in Rotated Sorted Array
6. Binary Search on Answer (e.g. minimize max, maximize minimum)
7. Find square root (using binary search)
8. Find nth root, etc.

Many
Variations
Same Idea !

7. Example : First Occurrence

```
int firstOccurrence(int[] nums, int target) {
   int left = 0, right = nums.length - 1;
   int ans = -1;
   while (left <= right) {
      int mid = left + (right - left) / 2;
      if (nums[mid] == target) {
         ans = mid;          // store and go left
         right = mid - 1;
      } else if (nums[mid] < target) {
         left = mid + 1;
      } else {
         right = mid - 1;
      }
   }
   return ans;
}
```

Find
First Occurrence
Efficiently !

8. Time & Space Complexity

Operation	Complexity
Time	O(log n)
Space	O(1) (iterative) / O(log n) (recursive)

Note :
Binary Search works only on sorted data or monotonic functions.

Better Code
Better Opportunities
Keep Going :)

Practice
Build
Improve :)

Think Practice Solve Improve :)

7. Sorting Patterns

Same Concepts. Different Questions. You Got This !

Small Steps Big Progress !

1. What is Sorting ?

- Sorting means arranging elements in a particular order (usually ascending or descending). It helps in searching, finding patterns and optimizing solutions.

2. Why Sorting ?

- Helps in finding duplicates
- Useful in two pointer problems
- Required in greedy algorithms
- Makes binary search possible
- Helps in range and interval problems

Sorted Data Simplifies Problems !

3. Types of Sorting

1. Built-in Sort (library function)
2. Custom Comparator Sorting
3. Sorting in Descending Order
4. Sorting based on a specific property (e.g. by second element, by length, etc.)

4. Basic Syntax (C++)

```cpp
#include <bits/stdc++.h>
using namespator std;

vector<int> v = {5, 1, 4, 2, 3};

// sort in ascending order
sort(v.begin(), v.end());

// sort in descending order
sort(v.begin(), v.end(), greater<int>());
```

5. When to Use Sorting ?

- When order of elements matters
- To find duplicates
- To use two pointer technique
- In greedy and interval problems
- When combined with binary search
- To process data efficiently

Sort Think Optimize Solve !

6. Common Sorting Patterns

1. Sort array in ascending / descending order
2. Custom comparator sorting (e.g. pair, struct)
3. Sort by multiple criteria (e.g. first then second)
4. Sort strings (lexicographical)
5. Sort based on frequency / value / length
6. Sort and then apply two pointers
7. Sort intervals (by start time)
8. Sort with custom logic (lambda / comparator)

7. Examples

1 Sort pairs by second element

```cpp
vector<pair<int,int>> v = {{1, 4}, {2, 1}, {3, 3}};
sort(v.begin(), v.end(), [](auto &a, auto &b) {
    return a.second < b.second;
});
```

2 Sort strings by length

```cpp
vector<string> v = {"apple", "cat", "banana"};
sort(v.begin(), v.end(), [](string &a, string &b) {
    return a.length() < b.length();
});
```

3 Sort in descending order

```cpp
vector<int> v = {5, 1, 4, 2, 3};
sort(v.begin(), v.end(), greater<int>());
// v = {5, 4, 3, 2, 1}
```

8. Time & Space Complexity

Algorithm	Time Complexity	Space Complexity
Built-in sort (C++ STL)	O(n log n)	O(1)
Custom comparator sort	O(n log n)	O(1)
Stable sort	O(n log n)	O(n)

9. Note :

Sorting is a powerful tool. Many complex problems become simple after sorting !

Better Code Better Opportunities Keep Going :)

Practice Build Improve :)

Learn
Practice
Solve
Improve :)

8. Stack + Monotonic Stack

Same Concepts. Different Questions. You Got This !

Small
Steps
Big
Progress !

1. What is Stack ?

Stack is a linear data structure that follows LIFO (Last In First Out) order.
The last element added is the first one to be removed.

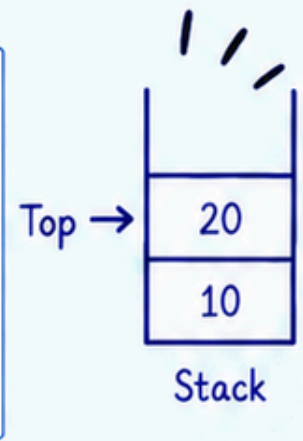
2. Basic Operations

Pusk() - Add an element to top
Pop() - Remove top element
Top() - Get top element
Empty() - Check if stack is empty

Simple but Very Useful !

3. Example

```
stack<int> st;
st.push(10);
st.push(20);
cout << st.top();  // 20
st.pop();
cout << st.top();  // 10
```



4. When to Use Stack ?

- When we need LIFO order
- In problems with nested structures (parentheses, brackets)
- Undo / Redo functionality
- Expression evaluation (infix, postfix, prefix)
- Used in many monotonic stack problems

5. Common Problems (Stack)

1. Valid Parentheses
2. Next Greater Element
3. Next Smaller Element
4. Stack Span Problem
5. Largest Rectangle in Histogram
6. Evaluate Reverse Polish Notation
7. Remove K Digits
8. Implement Queue using Stacks

Same Pattern Many Problems !

6. What is Monotonic Stack ?

Monotonic Stack is a stack that maintains its elements in either increasing or decreasing order.
It helps to find the next/previous greater or smaller element in O(n) time.

7. Types of Monotonic Stack

1. Increasing Stack – elements are in increasing order (used to find next smaller element)
2. Decreasing Stack – elements are in decreasing order (used to find next greater element)

Choose Direction Based on Problem !

8. Example : Next Greater Element (Monotonic Stack)

```
vector<int> nextGreater(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[st.top(1] < nums[i]) {
            res[st.top(1] = nums[i];
            st.pop();
        }
        st.push(i);
    }
    return res;
}
```

Use Stack to Maintain Candidates !

9. Dry Run (nums = [2, 1, 2, 4, 3])

i	nums[i]	Stack (indices)	res
0	2	[0]	[-1, -1, -1, -1, -1]
1	1	[0, 1]	[-1, -1, -1, -1, -1]
2	2	[2]	[2, -1, -1, -1, -1]
3	4	[3]	[2, 2, -1, -1, -1]
4	3	[3, 4]	[2, 2, -1, -1, -1]

10. Note :

Monotonic Stack helps us solve many problems in O(n) time which are hard to solve using brute force.

Better Code
Better Opportunities
Keep Going :)

Practice
Build
Improve :)

Learn
Practice
Solve
Improve :)

9. Queue + Deque

Same Concepts. Different Questions. You Got This !

Small
Steps
Big
Progress !

1. What is Queue ?

- Queue is a linear data structure that follows FIFO (First In First Out) order.
- The element inserted first is removed first.

First In
First Out
(FIFO)

2. Basic Operations (Queue)

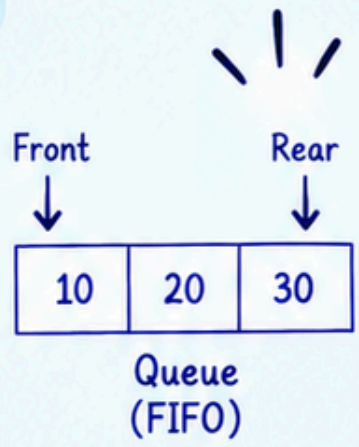
Operation	Description
enqueue()	Add an element to the rear
dequeue()	Remove an element from front
front()	Get the front element
back()	Get the rear element
size()	Number of elements
empty()	Check if queue is empty

Simple
Operations
Powerful
Structure !

3. Example (Queue)

```
queue<int> q;
q.push(10);
q.push(20);
q.push(30);

cout << q.front();  // 10
q.pop();
cout << q.front();  // 20
```



4. Common Problems (Queue)

1. Implement Queue using arrays / linked list
2. First non-repeating character in a stream
3. Sliding window maximum (using deque)
4. Rotten oranges (BFS)
5. Number of islands (BFS)
6. Task scheduling / CPU scheduling
7. Print first k elements of a queue
8. Implement a circular queue

Queue
is widely
used in BFS,
scheduling and
handling data
in FIFO order.

5. What is Deque ?

- Deque (Double Ended Queue) is a linear data structure that allows insertion and deletion from both ends (front and rear).
- It can be used as a queue, a stack, or both.

Insert
& Delete
grom Both
Ends !

6. Basic Operations (Deque)

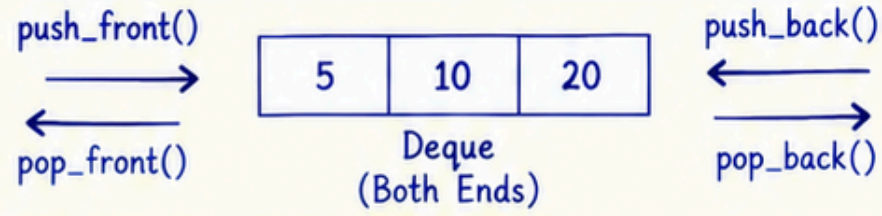
Operation	Description
push_front()	Insert at front
push_back()	Insert at rear
pop_front()	Delete from front
pop_back()	Delete from rear
front()	Get front element
back()	Get rear element
empty()	Check if deque is empty
size()	Number of elements

More
Flexible
Than Queue !

7. Example (Deque)

```
deque<int> dq;
dq.push_back(10);   // 10
dq.push_back(20);   // 10 20
dq.push_front(5);   // 5 10 20

cout << dq.front();  // 5
cout << dq.back();   // 20
dq.pop_front();      // 10 20
dq.pop_back();       // 10
```



8. Common Problems (Deque)

1. Sliding window maximum / minimum
2. LRU Cache
3. First negative number in every window of size k
4. Maximum of minimum for every window size
5. Palindrome check (using deque)
6. Implement stack using deque
7. Implement queue using deque

Deque is
very useful
in sliding window
and optimization
problems !

Better Code
Better Opportunities
Keep Going :)

Practice
Build
Improve :)

Learn
Practice
Solve
Improve :)

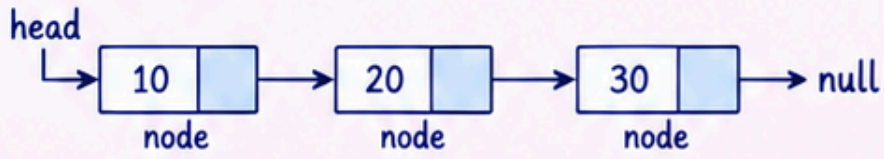
10. Linked List Patterns

Same Concepts. Different Questions. You Got This !

Small
Steps
Big
Progress !

1. What is a Linked List ?

- A Linked List is a linear data structure where elements (nodes) are stored in non-contiguous memory locations.
- Each node contains data and a reference (pointer) to the next node.



2. Types of Linked List

1. Singly Linked List (each node points to next)
2. Doubly Linked List (points to both next and previous)
3. Circular Linked List (last node points to head)

Different
Types for
Different
Problems !

3. Basic Node Structure (C++)

```cpp
struct Node {
    int val;
    Node* next;
    Node(int x) {
        val = x;
        next = nullptr;
    }
};
```

Simple
Structure,
Powerful
Usage !

4. Common Operations

Operation	Time Complexity
Insert at beginning	O(1)
Insert at end	O(n)
Delete a node	O(n)
Search a value	O(n)
Traverse	O(n)

5. Why Linked List ?

- Dynamic size (no need to specify size)
- Efficient insertions and deletions
- Useful in implementing stacks, queues, hash tables
- Used in many real world applications (LRU Cache, Undo/Redo, etc.)

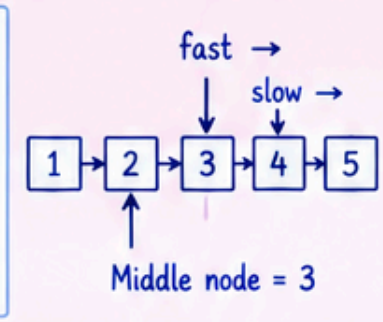
6. Common Linked List Patterns

1. Traverse the list
2. Find length of linked list
3. Find middle node (Slow & Fast pointer)
4. Reverse a linked list
5. Detect cycle (Floyd's algorithm)
6. Find start of cycle
7. Merge two sorted linked lists
8. Remove Nth node from end
9. Reorder list ($L_0 \to L_n \to L_1 \to L_{n-1} \ldots$)
10. Check if linked list is palindrome

Many
Patterns
One
Structure !

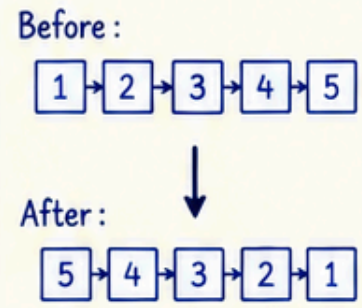
7. Pattern 1 : Find Middle Node (Slow & Fast Pointer)

```cpp
Node* middleNode(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```



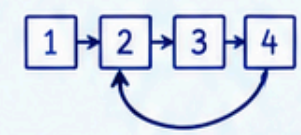
8. Pattern 2 : Reverse a Linked List

```cpp
Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```



9. Pattern 3 : Detect Cycle (Floyd's Algorithm)

```cpp
bool hasCycle(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```



Cycle detected
(slow == fast)

10. Note :

- Linked list problems are very common in interviews.
- Practice multiple patterns to build intuition.
- Most problems can be solved using simple pointer techniques.

Better Code
Better Opportunities
Keep Going :)

Practice
Build
Improve :)